ACCESIBILIDAD 360

Seguridad por diseño · Accesibilidad para todas las personas

AUDITORÍA DE SEGURIDAD DE APLICACIÓN WEB

Informe de Auditoría de Seguridad

Revisión white-box basada en OWASP Top 10 (2021)

Proyecto	Accesibilidad 360
Contexto académico	TFM Desarrollo con IA · Big School
Autor	Jorge Serna
Versión / fecha	2.0 (informe ampliado) · 9 de octubre de 2026
Metodología	Revisión estática white-box · OWASP Top 10 (2021)
Repositorio auditado	github.com/sernalovic/accesibilidad-360 · commit 5722e3e
Clasificación	CONFIDENCIAL · Uso restringido al equipo del proyecto y al tribunal evaluador
Valoración global	**ACEPTABLE para el alcance del TFM** · 0 críticas · 0 altas · 4 medias · 7 bajas · 1 informativa

Este documento contiene información sobre el estado de seguridad del proyecto. Su difusión fuera del ámbito indicado requiere autorización expresa del autor.

Índice de contenidos

1. Resumen ejecutivo 3
1.1 Indicadores clave 3
1.2 Distribución de los hallazgos 3
1.3 Mensajes principales 3
2. Objetivo 4
2.1 Objetivos específicos 4
3. Alcance 4
3.1 Fuera de alcance y limitaciones 4
4. Metodología 4
4.1 Escala de probabilidad e impacto 5
4.2 Niveles de severidad 5
5. Arquitectura de seguridad 6
5.1 Modelo de capas y puntos de hallazgo 6
5.2 Controles positivos observados 6
5.3 Flujo de autenticación y sesión 7
5.4 Cadena de entrega (CI/CD) 7
6. Evaluación según OWASP Top 10 (2021) 8
7. Resumen de hallazgos 8
8. Matriz de riesgos 10
8.1 Interpretación 10
8.2 Registro de riesgos 10
9. Detalle de hallazgos 11
H1 · Ausencia de limitación de intentos (rate limiting) 11
H2 · Duración elevada de la sesión 12
H3 · Ausencia de cabeceras HTTP de seguridad 13
H4 · Middleware sin protección de rutas (fail-open) 15
H5 · Recuperación de contraseña pendiente de implementar 16
H6 · Enumeración de usuarios en el registro 17
H7 · Longitudes máximas de entrada no acotadas 18
H8 · Validación de archivos subidos mejorable 19
H9 · Gestión de errores no homogénea 20
H10 · Endurecimiento del pipeline de CI y de las dependencias 21
H11 · Ausencia de auditoría de eventos de seguridad 22
H12 · Observaciones generales 22
10. Conclusiones ejecutivas 24
10.1 Valoración global 24
10.2 Fortalezas del proyecto 24
10.3 Debilidades y patrones comunes 24
10.4 Riesgo residual tras la remediación 24
10.5 Implicaciones para la decisión 25
10.6 Limitaciones de la conclusión 25
11. Recomendaciones priorizadas 26
11.1 Corto plazo (0–30 días) 26
11.2 Medio plazo (1–3 meses) 26
11.3 Largo plazo (3–12 meses) 27
11.4 Indicadores de éxito 27
Anexo A. Glosario 28
Anexo B. Lista de verificación para la reauditoría 28
Anexo C. Control del documento 28

1. Resumen ejecutivo

Se ha realizado una revisión estática (white-box) del proyecto **Accesibilidad 360**, que ha abarcado el código fuente, la arquitectura, la configuración, la documentación técnica y las pruebas automatizadas. El objetivo ha sido medir el nivel de seguridad de la aplicación antes de su despliegue y concretar qué debe corregirse y en qué orden.

> **Valoración global: ACEPTABLE**
> No se han identificado vulnerabilidades de severidad **Crítica** ni **Alta**. Los doce hallazgos son medidas de endurecimiento habituales antes de pasar a producción. El proyecto es **seguro para el alcance del TFM**; para una publicación abierta a usuarios reales deben cerrarse previamente los cuatro hallazgos de severidad Media (H1–H4).

1.1 Indicadores clave

0	0	4	7	1
CRÍTICOS	ALTOS	MEDIOS	BAJOS	INFORMATIVO

Puntuación máxima de riesgo: **12 sobre 25** (H1 y H3). Tres de las diez categorías OWASP presentan exposición Media (A05, A06 y A07). Cerrar los cuatro hallazgos Medios requiere unos **3–4 días** de trabajo, sin rediseño de arquitectura.

1.2 Distribución de los hallazgos

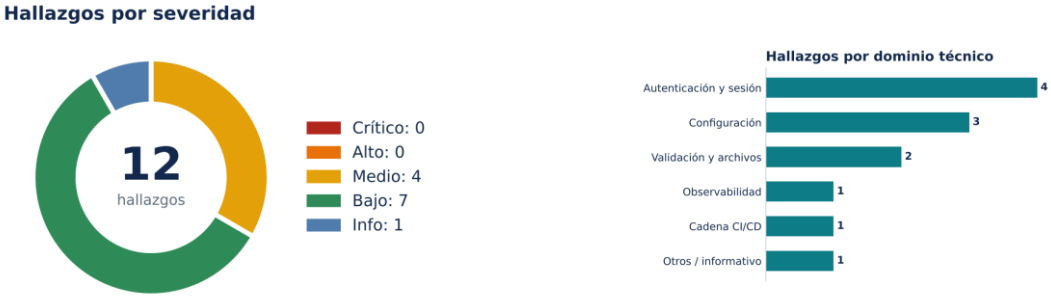


Figura 1. Hallazgos por severidad (izquierda) y por dominio técnico (derecha).

1.3 Mensajes principales

- **La base es sólida.** Validación en servidor, autorización centralizada y separación clara de responsabilidades reducen la superficie de ataque y explican la ausencia de hallazgos graves.
- **El riesgo se concentra en autenticación y sesión.** Cuatro de los doce hallazgos (H1, H2, H5, H6) afectan a ese dominio; es donde conviene invertir primero.
- **Los hallazgos de severidad Media son baratos de corregir.** Limitación de intentos, política de sesión, cabeceras HTTP y protección de rutas en el middleware suman unos pocos días de trabajo.
- **Faltan capacidades de detección.** Sin auditoría de eventos (H11) ni controles de seguridad en CI (H10), el proyecto no detectaría un incidente ni una regresión a tiempo.
- **La revisión es estática.** Antes de producción se recomienda una prueba dinámica y la verificación posterior de la remediación (H12).

2. Objetivo

El propósito de esta auditoría es evaluar el nivel de seguridad de la aplicación Accesibilidad 360 y detectar debilidades antes de su despliegue, proporcionando al equipo un diagnóstico claro, una priorización razonada y recomendaciones aplicables.

2.1 Objetivos específicos

- Identificar vulnerabilidades y carencias de endurecimiento en autenticación, autorización, validación, sesiones, configuración, dependencias y CI/CD.
- Valorar cada hallazgo con una escala homogénea de severidad, probabilidad, impacto y riesgo.
- Contrastar el proyecto con las diez categorías del OWASP Top 10 (2021).
- Proponer una hoja de ruta de remediación a corto, medio y largo plazo con responsables y criterios de aceptación.
- Dejar documentada la evidencia para que la corrección pueda verificarse de forma objetiva.

3. Alcance

La revisión cubre los dominios técnicos indicados en la tabla. Se examinaron el código fuente, la configuración y la documentación disponibles, así como los resultados de las pruebas automatizadas del proyecto.

Dominio	Elementos revisados	Estado
Autenticación	Registro, inicio de sesión, almacenamiento y verificación de credenciales, recuperación de cuenta	**Revisado**
Autorización	Control de acceso por rol y por recurso, modelo de permisos centralizado	**Revisado**
Validación de entradas	Esquemas de validación en servidor, longitudes, tipos, archivos subidos	**Revisado**
Gestión de sesiones	Emisión, caducidad, cookies, invalidación, middleware de sesión	**Revisado**
Configuración	Cabeceras HTTP, gestión de errores, secretos y variables de entorno	**Revisado**
Dependencias	npm audit sobre package-lock.json, versiones fijadas y proceso de actualización	**Revisado**
CI/CD	Definición del pipeline, permisos, controles automáticos	**Revisado**
Arquitectura	Capas, flujos de datos, separación de responsabilidades	**Revisado**

3.1 Fuera de alcance y limitaciones

- Pruebas dinámicas de penetración sobre un entorno desplegado (DAST) y fuzzing.
- Seguridad de la infraestructura de alojamiento y de los servicios de terceros.
- Ingeniería social, seguridad física y revisión legal del tratamiento de datos.
- Pruebas de carga y de resistencia a denegación de servicio distribuida.

Al tratarse de una revisión estática, algunos hallazgos se formulan como ausencia de control y cada ficha incluye un procedimiento para confirmarlos en pruebas. Cada hallazgo cita archivo y línea del repositorio (commit 5722e3e) y, cuando procede, un extracto literal del código; los fragmentos incluidos en las recomendaciones son propuestas, no código existente.

4. Metodología

El trabajo siguió un proceso en seis fases, de la planificación a la emisión del informe, tomando el OWASP Top 10 (2021) como marco de referencia y apoyándose en OWASP ASVS y en el catálogo CWE para la clasificación.

Fase	Actividad	Resultado
1	Planificación y alcance	Dominios, activos y criterios de valoración acordados
2	Revisión de arquitectura	Modelo lógico de capas y flujos de autenticación (apartado 5)
3	Revisión de código	Búsqueda de patrones inseguros en validación, sesiones y autorización
4	Revisión de configuración y CI	Cabeceras, errores, secretos, dependencias y pipeline
5	Clasificación y puntuación	Severidad, probabilidad, impacto y riesgo por hallazgo
6	Informe y recomendaciones	Este documento y la hoja de ruta de remediación

4.1 Escala de probabilidad e impacto

Cada hallazgo se puntúa de 1 a 5 en probabilidad (P) y en impacto (I). El riesgo se calcula como **R = P × I**, con un máximo de 25.

P	Probabilidad	Criterio	I	Impacto
1	Muy baja	Requiere condiciones excepcionales	1	Mínimo: sin efecto apreciable
2	Baja	Requiere atacante dedicado o varios fallos	2	Bajo: efecto local y reversible
3	Media	Posible con herramientas habituales	3	Moderado: afecta a una cuenta o función
4	Alta	Automatizable y frecuente	4	Alto: afecta a varias cuentas o datos
5	Muy alta	Trivial o ya explotado	5	Crítico: compromiso del sistema

4.2 Niveles de severidad

La severidad se deriva de la puntuación de riesgo y se representa en todo el documento con el mismo icono y color.

Severidad	Puntuación	Definición	Plazo orientativo
CRÍTICO	20 – 25	Explotación directa con compromiso grave del sistema o de los datos	Inmediato (≤ 72 h)
ALTO	15 – 19	Explotación probable con impacto significativo	≤ 15 días
MEDIO	8 – 14	Debilidad que requiere condiciones adicionales o con impacto acotado	≤ 30 días
BAJO	3 – 7	Medida de endurecimiento con efecto limitado	1 – 3 meses
INFORMATIVO	1 – 2	Observación sin riesgo directo	Planificable

5. Arquitectura de seguridad

Esta sección sitúa los hallazgos sobre el modelo lógico de la aplicación. Se trata de una vista simplificada y agnóstica de la tecnología, construida a partir de la revisión de la arquitectura y de la documentación técnica.

5.1 Modelo de capas y puntos de hallazgo

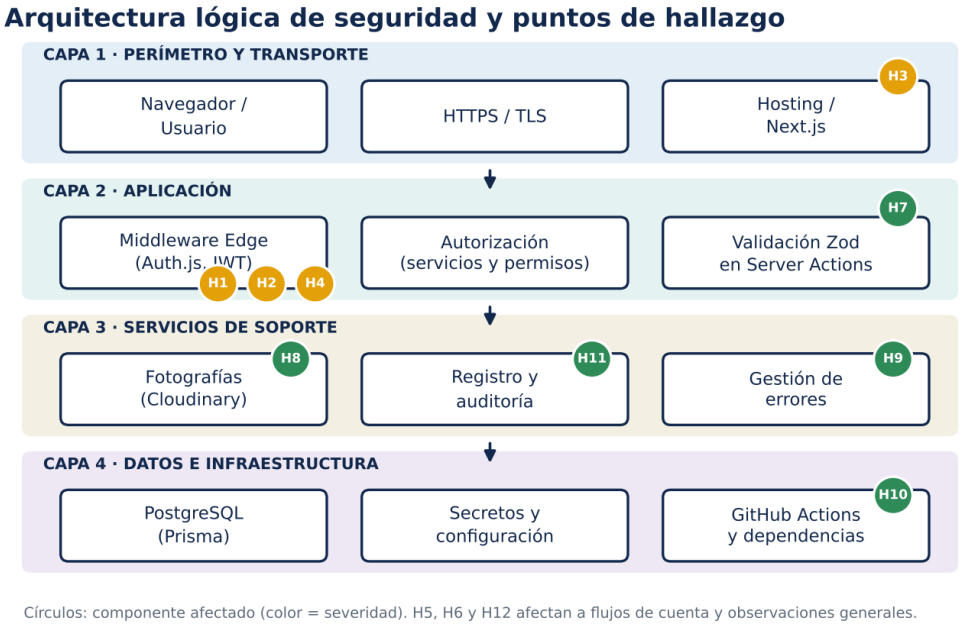


Figura 2. Arquitectura lógica de seguridad. Los círculos indican el componente afectado por cada hallazgo.

La aplicación se organiza en cuatro capas. Las capas 2 y 3 concentran la lógica de seguridad propia del proyecto y siete de los doce hallazgos; los de las capas 1 y 4 (H3, H10) se resuelven con configuración y proceso, no con código.

5.2 Controles positivos observados

La ausencia de hallazgos graves no es casual: obedece a decisiones de diseño acertadas que conviene preservar durante las correcciones.

Control	Observación (archivo:línea)	Valoración
Contraseñas	bcrypt con coste 12 (register.service.ts:7); mínimo de 12 caracteres con mayúscula, minúscula y número (register.schema.ts:5–10).	Adecuado
Login sin pistas	Mensaje genérico ante credenciales incorrectas (login.action.ts:25–27).	Adecuado
Validación en servidor	Zod en todas las Server Actions; userId siempre tomado de la sesión (create-establishment.action.ts:19–35).	Adecuado
Autorización por propietario	Regla única creador-o-admin comprobada contra el propietario persistido (lib/permissions.ts:17–19).	Adecuado
Acceso a datos	Prisma sin SQL crudo y sin dangerouslySetInnerHTML en todo el código; secretos solo en servidor y .env ignorado por Git.	Adecuado
Superficie exterior	Imágenes remotas limitadas a res.cloudinary.com (next.config.ts:10–16); única petición saliente a una URL fija (geocoding.service.ts:6).	Adecuado
Pruebas automatizadas	Amplia batería de pruebas unitarias y e2e con axe, pero sin casos negativos de seguridad.	Mejorable

5.3 Flujo de autenticación y sesión



Figura 3. Flujo de autenticación, sesión y autorización con los hallazgos asociados.

El flujo de autenticación concentra seis hallazgos: H1 y H7 en la entrada, H6 en el registro, H2 y H4 en la emisión y verificación de sesión, y H5 y H11 en los procesos de soporte. Conviene corregirlos de forma coordinada: comparten código y pruebas.

5.4 Cadena de entrega (CI/CD)

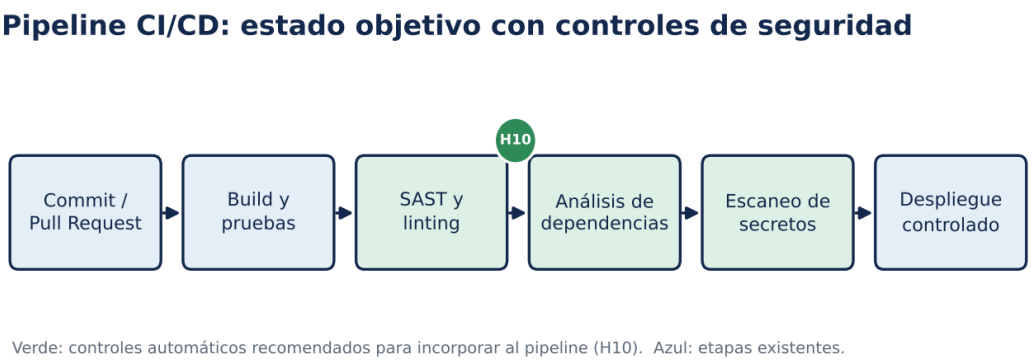


Figura 4. Pipeline objetivo con los controles de seguridad recomendados (H10).

El pipeline actual cubre construcción y pruebas. El estado objetivo añade tres controles automáticos (análisis estático, dependencias y secretos) con un coste de mantenimiento muy bajo una vez configurados.

6. Evaluación según OWASP Top 10 (2021)

La tabla resume la exposición de la aplicación en cada categoría. El nivel «Medio» aparece en tres categorías (A05, A06 y A07); el resto es Bajo, salvo A10, que no aplica al diseño actual.

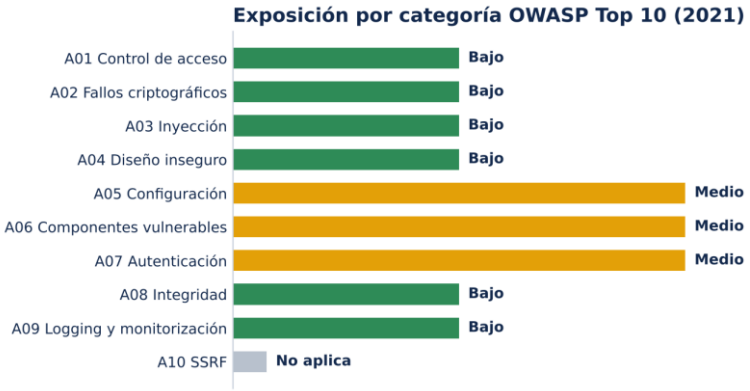


Figura 5. Exposición por categoría del OWASP Top 10 (2021).

ID	Categoría	Riesgo	Hallazgos	Comentario
A01	Control de acceso roto	Bajo	H4 (riesgo de regresión)	Layout protegido y auth() en cada acción; propietario o admin desde servicios.
A02	Fallos criptográficos	Bajo	H3 (HSTS)	bcrypt y JWT firmado con AUTH_SECRET; falta forzar HTTPS (HSTS).
A03	Inyección	Bajo	H7, H8	Zod y Prisma sin SQL crudo; faltan longitudes máximas y firma de archivo.
A04	Diseño inseguro	Bajo	H5, H7, H8	Arquitectura sólida; carencias puntuales de diseño.
A05	Configuración de seguridad incorrecta	Medio	H3, H4, H9	Cabeceras ausentes, fail-open y errores no homogéneos.
A06	Componentes vulnerables y obsoletos	Medio	H10	npm audit: 18 vulnerabilidades (2 críticas, en herramientas de desarrollo).
A07	Fallos de identificación y autenticación	Medio	H1, H2, H5, H6	Sin límite de intentos, sesión de 30 días y rol cacheado en el JWT.
A08	Fallos de integridad de software y datos	Bajo	H8, H10	Pipeline sin controles y subidas validadas por tipo declarado.
A09	Fallos de registro y monitorización	Bajo	H9, H11	Sin auditoría estructurada de eventos.
A10	Falsificación de peticiones del lado servidor (SSRF)	No aplica	—	Única petición saliente a Nominatim con URL fija; el usuario solo aporta la consulta.

Lectura. La concentración de exposición Media en A05, A06 y A07 es coherente con un proyecto que ha puesto el esfuerzo en la lógica de negocio y que aún no ha pasado por una fase de endurecimiento: son las categorías que dependen de configuración, de proceso y de automatización más que del diseño.

7. Resumen de hallazgos

Registro consolidado de los doce hallazgos ordenados por puntuación de riesgo. El detalle completo de cada uno se encuentra en el apartado 9.

ID	Hallazgo	Severidad	P	I	R	Prioridad	Plazo
H1	Ausencia de limitación de intentos (rate limiting)	Medio	4	3	**12**	Alta	Corto plazo

ID	Hallazgo	Severidad	P	I	R	Prioridad	Plazo
H3	Ausencia de cabeceras HTTP de seguridad	Medio	4	3	12	Alta	Corto plazo
H2	Duración elevada de la sesión	Medio	3	3	9	Alta	Corto plazo
H4	Middleware sin protección de rutas (fail-open)	Medio	2	4	8	Alta	Corto plazo
H5	Recuperación de contraseña pendiente de implementar	Bajo	3	2	6	Media	Medio plazo
H6	Enumeración de usuarios en el registro	Bajo	3	2	6	Media	Medio plazo
H8	Validación de archivos subidos mejorable	Bajo	2	3	6	Media	Medio plazo
H9	Gestión de errores no homogénea	Bajo	3	2	6	Media	Medio plazo
H10	Endurecimiento del pipeline de CI y de las dependencias	Bajo	2	3	6	Media	Medio plazo
H7	Longitudes máximas de entrada no acotadas	Bajo	2	2	4	Media	Medio plazo
H11	Ausencia de auditoría de eventos de seguridad	Bajo	2	2	4	Media	Medio plazo
H12	Observaciones generales	Info	1	1	1	Baja	Largo plazo

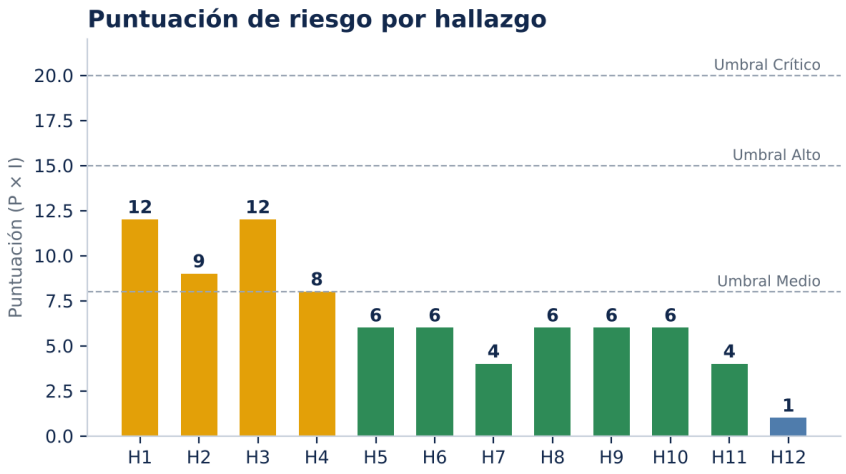


Figura 6. Puntuación de riesgo (P × I) de cada hallazgo y umbrales de severidad.

Cuatro hallazgos superan el umbral de severidad Media (8 puntos) y los ocho restantes quedan por debajo. Ninguno se acerca a los umbrales Alto (15) o Crítico (20), lo que confirma que no existen vulnerabilidades graves en el alcance revisado.

8. Matriz de riesgos

La matriz sitúa cada hallazgo según su probabilidad y su impacto. El color de cada celda corresponde a la banda de severidad de su puntuación (P × I); las celdas ocupadas aparecen en color intenso y muestran los identificadores de los hallazgos.

IMPACTO	1 · Muy baja	2 · Baja	3 · Media	4 · Alta	5 · Muy alta
5 · Crítico	5	10	15	20	25
4 · Alto	4	H4 8	12	16	20
3 · Moderado	3	H8 · H10 6	H2 9	H1 · H3 12	15
2 · Bajo	2	H7 · H11 4	H5 · H6 · H9 6	8	10
1 · Mínimo	H12 1	2	3	4	5

PROBABILIDAD

CRÍTICO 20–25 | ALTO 15–19 | MEDIO 8–14 | BAJO 3–7 | INFORMATIVO 1–2

8.1 Interpretación

- **Zona Media (8–14):** H1, H3, H2 y H4. H1 y H3 se sitúan en probabilidad alta; H4 combina baja probabilidad con impacto alto, lo que justifica su corrección pese a su puntuación ajustada.
- **Zona Baja (3–7):** H5, H6, H9 (P=3, I=2), H8 y H10 (P=2, I=3) y H7 y H11 (P=2, I=2). Son medidas de endurecimiento sin efecto grave aislado.
- **Zonas Alta y Crítica:** vacías. No existe ningún hallazgo con probabilidad o impacto que combinados alcancen 15 puntos.

8.2 Registro de riesgos

ID	Riesgo asociado	Riesgo actual	Objetivo	Tratamiento	Condición de cierre
H1	Fuerza bruta y relleno de credenciales	Medio	Bajo (≤ 4)	Mitigar	429 tras el umbral y espera progresiva por cuenta
H3	XSS sin contención, clickjacking, degradación a HTTP	Medio	Bajo (≤ 4)	Mitigar	Cabeceras presentes y comprobadas en CI
H2	Secuestro de sesión con ventana amplia	Medio	Bajo (≤ 4)	Mitigar	Caducidad por inactividad y absoluta aplicada
H4	Ruta nueva sin protección con el borde abierto	Medio	Bajo (≤ 4)	Eliminar	Denegación por defecto y pruebas de acceso
H5–H12	Endurecimiento general y madurez (ocho hallazgos)	Bajo	Mínimo (≤ 2)	Mitigar	Controles implementados y verificados

9. Detalle de hallazgos

Cada hallazgo se documenta con una ficha homogénea: descripción, evidencia, impacto, probabilidad, riesgo, recomendación y prioridad. Los hallazgos de severidad Media (H1–H4) se detallan con más extensión por ser los que condicionan la puesta en producción.

H1 · Ausencia de limitación de intentos (rate limiting)

Severidad	⚠ **MEDIO**	**Riesgo (P × I)**	**4 × 3 = 12** de 25
Probabilidad	4 · Alta	**Impacto**	3 · Moderado
Categoría OWASP	A07 · Fallos de identificación y autenticación	**CWE**	CWE-307 · CWE-770
Componente	Server Actions de login y registro	**Prioridad**	Alta · Corto plazo

Descripción

Ni el login ni el registro limitan cuántas peticiones puede enviar un mismo origen o una misma cuenta. Cada intento se procesa completo, incluida la comparación bcrypt, sin contadores, esperas progresivas ni desafíos adicionales.

La especificación del proyecto deja fuera de forma explícita el bloqueo de cuentas y la auditoría de accesos (SPEC-010, líneas 396–397), de modo que la ausencia es una decisión de alcance documentada y no un olvido. Aun así, es la carencia con mayor puntuación de riesgo del informe.

Evidencia

login.action.ts:12–35 invoca signIn sin control previo; **lib/auth/auth.ts:25–31** delega en verifyCredentials, que ejecuta compare de bcrypt en cada intento (**login.service.ts:25**). **register.action.ts:10–37** tampoco limita las altas, y el middleware está vacío (**middleware.ts:12–14**).

El coste de bcrypt es 12 (**register.service.ts:7**), unos cientos de milisegundos de CPU por intento, lo que convierte el endpoint de login también en un amplificador de denegación de servicio.

1. En un entorno de pruebas, lanzar una ráfaga de 50 intentos de login con contraseña incorrecta sobre la misma cuenta.
2. Comprobar que todos se procesan con el mismo resultado y sin espera ni bloqueo.

```
// features/auth/services/login.service.ts:20-25
const user = await prisma.user.findUnique({ where: { email } });
if (!user?.passwordHash) { return null; }
const valid = await compare(password, user.passwordHash);
```

Impacto

Permite fuerza bruta y relleno de credenciales contra cuentas reales. La política de contraseñas (mínimo 12 caracteres, **register.schema.ts:5–10**) reduce mucho la probabilidad de éxito, pero no impide el abuso del servicio: cada intento consume CPU y las altas masivas llenan la base de datos.

Las subidas de fotografías y la geocodificación son acciones autenticadas igualmente sin cuota, con coste en Cloudinary y en Nominatim.

Confidencialidad	Integridad	Disponibilidad
Alto	**Medio**	**Medio**

Probabilidad

Alta (4/5). El formulario será público y expuesto a tráfico automatizado desde el primer día; las herramientas necesarias son gratuitas y la ausencia de límite es total.

Riesgo

Riesgo = 4 × 3 = 12 (Medio). La contraseña robusta y el mensaje genérico de login moderan el impacto, pero no la probabilidad.

INFORMATIVO 1–2	BAJO 3–7	▶ MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Aplicar limitación por capas a login, registro, cambio de contraseña y subidas:

- Límite por IP y por correo en un almacén compartido (por ejemplo Upstash Ratelimit o una tabla en la base de datos), porque la memoria de proceso no funciona en despliegues serverless.
- Espera progresiva tras 5 fallos en 15 minutos por cuenta, evitando el bloqueo permanente para no facilitar la denegación dirigida.
- Cuota de subidas por usuario y por hora, y límite de altas por IP.
- Registrar cada bloqueo como evento de seguridad (H11).

```
// features/auth/actions/login.action.ts (al inicio de loginUserAction)
const { success } = await ratelimit.limit(`login:${ip}:${parsed.data.email}`);
if (!success) return { success: false, message: "Demasiados intentos. Inténtalo más tarde." };
```

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Alta	Corto plazo (0–30 días)	Bajo (1 día)	Desarrollo backend

Criterio: Mayor riesgo del informe, solución conocida y acotada.

H2 · Duración elevada de la sesión

Severidad	⚠ MEDIO	**Riesgo (P × I)**	**3 × 3 = 9** de 25
Probabilidad	3 · Media	**Impacto**	3 · Moderado
Categoría OWASP	A07 · Fallos de identificación y autenticación	**CWE**	CWE-613 · CWE-266
Componente	Sesión JWT (Auth.js)	**Prioridad**	Alta · Corto plazo

Descripción

La sesión usa JWT sin parámetros de caducidad propios, por lo que rige el valor por defecto de Auth.js: 30 días de vigencia absoluta. No hay caducidad por inactividad ni revocación en servidor.

Además, el rol del usuario se copia al token en el inicio de sesión y no se vuelve a leer: un cambio de rol, la retirada de privilegios de un administrador o el borrado de una cuenta no surten efecto hasta que el token expira.

Evidencia

lib/auth/auth.config.ts:11 fija únicamente la estrategia (strategy: "jwt"), sin maxAge ni updateAge. **lib/auth/auth.constants.ts:1–4** está vacío, con el comentario de que los tiempos de expiración llegarán en fases posteriores.

lib/auth/auth.ts:36–41 guarda el rol en el token solo cuando existe user, es decir, en el login. **profile.service.ts** (changePassword) cambia el hash pero, al ser la sesión stateless, no puede invalidar los tokens ya emitidos.

1. Iniciar sesión y descodificar la cookie de sesión para leer el campo exp (≈ 30 días).
2. Cambiar el rol del usuario en la base de datos y comprobar que la sesión activa conserva el rol anterior.

```
// lib/auth/auth.config.ts:11
session: { strategy: "jwt" },   // sin maxAge ni updateAge
```

Impacto

Un token robado (equipo compartido, extensión maliciosa, XSS futuro) sirve durante hasta 30 días y no puede revocarse. Un administrador degradado o una cuenta eliminada conserva el acceso mientras dure el token.

El impacto queda limitado a la cuenta afectada; las comprobaciones de propietario se hacen en servidor con el id del token (**lib/permissions.ts:17–19**).

Confidencialidad	Integridad	Disponibilidad
Alto	**Medio**	**Bajo**

Probabilidad

Media (3/5). Requiere un vector previo para obtener el token, pero la vigencia de un mes amplifica cualquier incidente y hace probable que haya sesiones olvidadas en dispositivos ajenos.

Riesgo

Riesgo = 3 × 3 = 9 (Medio). Es un multiplicador de otros ataques más que una vulnerabilidad aislada.

INFORMATIVO 1–2	BAJO 3–7	▶ MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Acotar la vigencia y reintroducir un punto de control en servidor:

- Fijar maxAge de 8 horas (o 24 como máximo) y updateAge para renovar la sesión solo con actividad.
- Releer el rol (y la existencia del usuario) desde la base de datos con una frecuencia acotada, por ejemplo cada 15 minutos, dentro del callback jwt.
- Añadir un campo tokenVersion al usuario, incrementarlo al cambiar la contraseña y comprobarlo en el callback jwt; o migrar a sesiones de base de datos con el adaptador Prisma que ya está en las dependencias.
- Mantener las cookies HttpOnly, Secure y SameSite=Lax que Auth.js emite por defecto.

```
session: { strategy: "jwt", maxAge: 60 * 60 * 8, updateAge: 60 * 30 },
```

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Alta	Corto plazo (0–30 días)	Bajo-medio (1 día)	Desarrollo backend

Criterio: Reduce la ventana de ataque con un cambio de configuración.

H3 · Ausencia de cabeceras HTTP de seguridad

Severidad	⚠ **MEDIO**	**Riesgo (P × I)**	**4 × 3 = 12** de 25
Probabilidad	4 · Alta	**Impacto**	3 · Moderado
Categoría OWASP	A05 · Configuración de seguridad incorrecta	**CWE**	CWE-693 · CWE-1021
Componente	next.config.ts y respuesta HTTP	**Prioridad**	Alta · Corto plazo

Descripción

La aplicación no envía ninguna de las cabeceras que activan las defensas del navegador: política de seguridad de contenido (CSP), HSTS, protección frente a incrustación en marcos, X-Content-Type-Options, Referrer-Policy ni Permissions-Policy.

No son vulnerabilidades en sí, pero actúan como segunda línea de defensa. Con CSP, un XSS futuro tendría un alcance mucho menor; con frame-ancestors se impide el clickjacking sobre las acciones de borrado y valoración.

Evidencia

next.config.ts:3–18 solo define serverActions.bodySizeLimit e images.remotePatterns; no existe la función headers(). El middleware tampoco añade cabeceras (**middleware.ts:12–14**).

Los orígenes de terceros que habrá que permitir en la CSP son visibles en el código: imágenes de Cloudinary (**next.config.ts:12–13**) y teselas de OpenStreetMap (**MapFrame.tsx:26**).

1. Ejecutar curl -sI sobre la URL desplegada y buscar las seis cabeceras: no aparece ninguna.
2. Contrastar el resultado con un analizador público (Mozilla Observatory).

```
// next.config.ts:3-18 (extracto)
const nextConfig: NextConfig = {
  experimental: { serverActions: { bodySizeLimit: "5mb" } },
  images: { remotePatterns: [{ protocol: "https", hostname: "res.cloudinary.com" }] },
};   // sin headers()
```

Impacto

Sin CSP, cualquier inyección de script se ejecuta sin restricción. Sin HSTS, un atacante en la red puede forzar conexiones sin cifrar en el primer acceso. Sin frame-ancestors, la aplicación puede incrustarse en una página maliciosa.

Al ser una aplicación de accesibilidad, la CSP debe probarse con ayudas técnicas y con el test axe existente (**tests/e2e/accessibility.spec.ts**) antes de pasar a modo bloqueante.

Confidencialidad	Integridad	Disponibilidad
Medio	**Medio**	**Bajo**

Probabilidad

Alta (4/5). La ausencia es permanente y afecta a todas las respuestas; lo que condiciona la explotación es la aparición de otro fallo, pero la superficie queda abierta desde el primer día.

Riesgo

Riesgo = 4 × 3 = 12 (Medio). Comparte la puntuación máxima con H1 y es la corrección con mejor relación coste-beneficio de la auditoría.

INFORMATIVO 1–2	BAJO 3–7	▶ MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Añadir las cabeceras desde next.config.ts y endurecer la CSP de forma iterativa:

- Empezar con Content-Security-Policy-Report-Only; permitir img-src a res.cloudinary.com y *.tile.openstreetmap.org; pasar a modo bloqueante tras una semana sin violaciones.
- Usar nonces generados en el middleware para los scripts en línea de Next.js, aprovechando que el middleware ya existe.
- HSTS con max-age de 6 meses y ampliar después a 1 año.
- Añadir una prueba que falle si falta alguna cabecera obligatoria.

```
const csp = ["default-src 'self'", "frame-ancestors 'none'",
  "img-src 'self' data: https://res.cloudinary.com https://*.tile.openstreetmap.org"].join("; ");
async headers() {
  return [{ source: "/:path*", headers: [
    { key: "Strict-Transport-Security", value: "max-age=15552000; includeSubDomains" },
    { key: "X-Content-Type-Options", value: "nosniff" },
    { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
    { key: "Content-Security-Policy-Report-Only", value: csp },
  ]}];
},
```

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Alta	Corto plazo (0–30 días)	Bajo (1 día)	Desarrollo / DevOps

Criterio: Máxima relación coste-beneficio; protege toda la aplicación.

H4 · Middleware sin protección de rutas (fail-open)

Severidad	MEDIO	**Riesgo (P × I)**	**2 × 4 = 8** de 25
Probabilidad	2 · Baja	**Impacto**	4 · Alto
Categoría OWASP	A05 · Configuración incorrecta · A01 (riesgo de regresión)	**CWE**	CWE-636 · CWE-862
Componente	middleware.ts y auth.config.ts	**Prioridad**	Alta · Corto plazo

Descripción

El middleware está montado pero es intencionalmente un paso libre: ejecuta la comprobación de sesión y deja pasar cualquier petición. El callback authorized devuelve siempre true. Es un comportamiento fail-open: el borde de la aplicación admite todo por defecto y delega la protección en capas posteriores.

Hoy esas capas funcionan: el layout del grupo protegido exige sesión y cada Server Action vuelve a comprobarla. El riesgo es de regresión, porque una ruta nueva fuera del grupo protegido, o un cambio en el layout, quedaría abierta sin que nada lo detecte.

Evidencia

middleware.ts:4–14 documenta que solo refresca la sesión y deja un TODO; **lib/auth/auth.config.ts:13–18** define authorized() { return true } con otro TODO; y **tests/lib/auth-config.test.ts:13–19** fija ese comportamiento con una prueba ("no protege rutas todavía").

La protección real descansa en **app/(protected)/layout.tsx:10** (requireSession, un único punto) y en la comprobación auth() de las 13 Server Actions que requieren sesión. Páginas como establishments/new dependen solo del layout ("Protegida por el layout"). Next.js desaconseja basar el control de acceso únicamente en layouts.

1. Solicitar sin sesión una ruta del grupo protegido: el layout redirige a /login (comportamiento correcto).
2. Crear una página de prueba fuera de (protected) y comprobar que el middleware no la bloquea.

```
// lib/auth/auth.config.ts:13-18
callbacks: {
  authorized() {
    // TODO(SPEC-010, fase autorización): proteger rutas por autenticación y rol.
    return true;
  },
},
```

Impacto

Si en el futuro se añade una página o un route handler sin comprobación propia, o se refactoriza el layout, quedaría accesible sin autenticar. El panel /admin ya tiene un guard propio (**admin/page.tsx:17–20**), pero es una defensa por página, no por política.

Por eso el impacto potencial es alto aunque la probabilidad actual sea baja.

Confidencialidad	Integridad	Disponibilidad
Alto	**Alto**	**Bajo**

Probabilidad

Baja (2/5). Hoy no hay una ruta expuesta, pero el proyecto crece por fases y el propio código marca la protección como tarea pendiente.

Riesgo

Riesgo = 2 × 4 = 8 (Medio, en el límite inferior). Se mantiene como Medio por la gravedad potencial y por ser un patrón de seguridad que conviene cerrar antes de producción.

INFORMATIVO 1–2	BAJO 3–7	▶ MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Completar la fase de autorización pendiente y convertirla en denegación por defecto:

- Implementar authorized() para exigir sesión en todo salvo una lista explícita de rutas públicas (/, /login, /register, /legal, /privacy, /cookies).
- Mantener requireSession en el layout y auth() en las acciones como segunda barrera (defensa en profundidad).
- Aplicar el control por rol de /admin también en el middleware.
- Sustituir la prueba actual por pruebas que verifiquen que una ruta privada sin sesión se deniega.

```
authorized({ auth, request }) {
  const isPublic = PUBLIC_PATHS.some((p) => request.nextUrl.pathname === p);
  return isPublic || !!auth?.user;   // denegar por defecto
},
```

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Alta	Corto plazo (0–30 días)	Bajo-medio (1 día)	Desarrollo backend

Criterio: Cierra una tarea ya identificada con impacto potencial alto.

H5 · Recuperación de contraseña pendiente de implementar

Severidad	BAJO	**Riesgo (P × I)**	**3 × 2 = 6** de 25
Probabilidad	3 · Media	**Impacto**	2 · Bajo
Categoría OWASP	A07 · Fallos de identificación y autenticación	**CWE**	CWE-640
Componente	Gestión de cuentas	**Prioridad**	Media · Medio plazo

Descripción

No existe flujo de autoservicio para recuperar la contraseña. La especificación SPEC-010 sí lo define (líneas 54, 100–102 y 195, con casos de token inválido o expirado en la 272), por lo que está planificado y pendiente.

El riesgo está en cómo se implemente: los flujos de recuperación improvisados son una fuente clásica de toma de cuentas (tokens predecibles, reutilizables o sin caducidad, y enumeración).

Evidencia

La búsqueda de rutas, formularios y servicios no encuentra funcionalidad de recuperación: **app/(auth)** solo contiene login y register, y no hay referencias a «olvidé» o «forgot» en la interfaz. Existe cambio de contraseña con sesión iniciada (**change-password.action.ts**), que no cubre este caso.

Impacto

Pérdida de acceso para usuarios legítimos y carga de soporte manual. Si se añade con prisas, puede introducir un camino de toma de cuenta más débil que el login. La especificación excluye la verificación de correo, lo que condiciona el diseño.

Confidencialidad	Integridad	Disponibilidad
Bajo	Bajo	Medio

Probabilidad

Media (3/5): cualquier usuario olvida su contraseña tarde o temprano.

Riesgo

Riesgo = 3 × 2 = 6 (Bajo).

INFORMATIVO 1–2	▶ BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Implementarlo siguiendo la hoja de OWASP para recuperación de contraseña:

- Token aleatorio de al menos 128 bits, de un solo uso, caducidad de 15–30 minutos y guardado con hash en una tabla propia.
- Respuesta idéntica exista o no la cuenta y limitación de intentos (H1).
- Al completar el restablecimiento, invalidar las sesiones activas (tokenVersion, H2), notificar por correo y registrar el evento (H11).

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Media	Medio plazo (1–3 meses)	Medio (2–3 días)	Desarrollo full-stack

Criterio: Funcionalidad esperada; su seguridad depende del diseño.

H6 · Enumeración de usuarios en el registro

Severidad	▼ BAJO	**Riesgo (P × I)**	**3 × 2 = 6** de 25
Probabilidad	3 · Media	**Impacto**	2 · Bajo
Categoría OWASP	A07 · Fallos de identificación y autenticación	**CWE**	CWE-204 · CWE-203
Componente	Registro y login	**Prioridad**	Media · Medio plazo

Descripción

El registro responde de forma distinta cuando el correo ya existe, y además devuelve el error asociado al campo de correo. Cualquiera puede comprobar si una persona concreta tiene cuenta.

El login sí es cuidadoso con el mensaje genérico, pero su tiempo de respuesta delata la existencia del usuario, como detalla la evidencia.

Evidencia

register.service.ts:28–31 lanza EmailAlreadyExistsError si el correo existe y **register.action.ts:26–31** devuelve «El correo electrónico ya está registrado.» como error del campo email.

En el login, **login.service.ts:21–23** retorna null sin ejecutar bcrypt cuando el usuario no existe, mientras que con usuario existente se ejecuta compare (**línea 25**, coste 12): la diferencia de tiempo es medible.

1. Registrar un correo nuevo y otro ya existente y comparar mensaje y código de respuesta.
2. Medir el tiempo de un login fallido con correo inexistente frente a uno existente.

Impacto

Permite confirmar si una persona usa el servicio y construir listas de cuentas válidas para fuerza bruta (H1) o phishing. Al tratarse de una plataforma de accesibilidad, puede revelar información sensible sobre el perfil de las personas usuarias, por lo que conviene tratarlo con cuidado desde el RGPD.

Confidencialidad	Integridad	Disponibilidad
Medio	**Bajo**	**Bajo**

Probabilidad

Media (3/5): es trivial de automatizar, aunque su valor aislado es limitado.

Riesgo

Riesgo = 3 × 2 = 6 (Bajo).

INFORMATIVO 1–2	▶ BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Unificar la respuesta y el tiempo en ambos flujos:

- Mostrar siempre el mismo mensaje en el registro y enviar al correo existente un aviso de intento de alta (requiere el envío de correo previsto en H5).
- En el login, ejecutar un compare contra un hash ficticio cuando el usuario no existe para igualar los tiempos.

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Media	Medio plazo (1–3 meses)	Bajo (0,5 día)	Desarrollo backend

Criterio: Reduce fuga de información con cambio localizado.

H7 · Longitudes máximas de entrada no acotadas

Severidad	BAJO	**Riesgo (P × I)**	**2 × 2 = 4** de 25
Probabilidad	2 · Baja	**Impacto**	2 · Bajo
Categoría OWASP	A04 · Diseño inseguro	**CWE**	CWE-770 · CWE-20
Componente	Esquemas de validación (Zod)	**Prioridad**	Media · Medio plazo

Descripción

La validación en servidor es sistemática y usa Zod, pero varios campos de texto no tienen longitud máxima. Los nombres sí están acotados a 100 caracteres, lo que muestra que el criterio existe y falta aplicarlo al resto.

Evidencia

Sin máximo: contraseña en registro (**register.schema.ts:5–10**) y en login (**login.schema.ts:12–14**), dirección (**establishment.schema.ts:17–20**), descripción (**línea 29**) y comentario de valoración (**review.schema.ts:32**). El correo tampoco tiene tope.

El único límite global es el tamaño del cuerpo de las Server Actions de 5 MB (**next.config.ts:5–7**), que mitiga pero no sustituye a los límites por campo. Además, bcrypt solo considera los primeros 72 bytes de la contraseña.

Impacto

Consumo excesivo de CPU (bcrypt con contraseñas enormes), memoria y almacenamiento, y registros desproporcionados en la base de datos. Efecto principal sobre la disponibilidad.

Confidencialidad	Integridad	Disponibilidad
Bajo	**Bajo**	**Medio**

Probabilidad

Baja (2/5): exige un usuario deliberadamente abusivo y el beneficio es solo de disponibilidad.

Riesgo

Riesgo = 2 × 2 = 4 (Bajo).

INFORMATIVO 1–2	▶ BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Acotar todos los campos en los esquemas:

- Contraseña: .max(72) (límite real de bcrypt) en registro, login y cambio de contraseña; correo .max(254).
- Dirección .max(200), descripción .max(2000) y comentario .max(1000), alineados con la interfaz.
- Añadir una prueba por esquema que verifique el rechazo de entradas sobredimensionadas.

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Media	Medio plazo (1–3 meses)	Bajo (0,5 día)	Desarrollo backend

Criterio: Cambio sistemático y de bajo riesgo.

H8 · Validación de archivos subidos mejorable

Severidad	BAJO	**Riesgo (P × I)**	**2 × 3 = 6** de 25
Probabilidad	2 · Baja	**Impacto**	3 · Moderado
Categoría OWASP	A04 · Diseño inseguro · A08 · Integridad	**CWE**	CWE-434 · CWE-345
Componente	Subida de fotografías	**Prioridad**	Media · Medio plazo

Descripción

La validación de las fotografías confía en el tipo MIME que declara el cliente. No se comprueba la firma real del archivo ni existe una cuota de subida por usuario.

El riesgo real es bajo porque los archivos nunca se interpretan en el servidor: se reenvían a Cloudinary, que solo acepta imágenes decodificables.

Evidencia

photo.schema.ts:11–14 acepta el archivo si file.type está en jpeg, png o webp, valor controlado por el cliente. **photo.service.ts:81–86** convierte el archivo en buffer y lo sube; solo después comprueba las dimensiones mínimas y borra la imagen si no las cumple.

El límite es de 10 fotografías por establecimiento (**photo.service.ts:76–79**), no por usuario, y la acción exige sesión (**upload-photo.action.ts:20–24**). Mitigación existente: resource_type "image" en Cloudinary (**photo-storage.service.ts:27–29**).

Impacto

Subida de contenido no deseado con tipo falsificado, consumo de la cuota de Cloudinary y trabajo innecesario (subir y borrar imágenes pequeñas). No hay ejecución de código al no interpretarse el contenido en servidor.

Confidencialidad	Integridad	Disponibilidad
Medio	**Medio**	**Bajo**

Probabilidad

Baja (2/5): exige un usuario autenticado con intención abusiva.

Riesgo

Riesgo = 2 × 3 = 6 (Bajo).

INFORMATIVO 1–2	▶ BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Validar el contenido y limitar el abuso:

- Comprobar los primeros bytes (firma) del archivo frente a JPEG, PNG y WebP antes de subirlo.
- Cuota de subidas por usuario y por hora (compartiendo el limitador de H1).

- Mover las restricciones de tamaño y formato a un upload preset de Cloudinary, para que se apliquen también en el servicio externo.

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Media	Medio plazo (1–3 meses)	Medio (1–2 días)	Desarrollo backend

Criterio: Controla un vector clásico antes de que crezca el uso.

H9 · Gestión de errores no homogénea

Severidad	BAJO	**Riesgo (P × I)**	**3 × 2 = 6** de 25
Probabilidad	3 · Media	**Impacto**	2 · Bajo
Categoría OWASP	A05 · Configuración incorrecta · A09	**CWE**	CWE-209 · CWE-391
Componente	Server Actions y páginas	**Prioridad**	Media · Medio plazo

Descripción

Las Server Actions devuelven mensajes genéricos al usuario, lo cual es correcto, pero el tratamiento es desigual: algunas acciones registran el error y la mayoría lo descarta sin dejar rastro. No hay páginas de error propias para fallos inesperados.

Evidencia

Solo existe un console.error en todo el código de aplicación (**upload-photo.action.ts:47**). Los bloques catch de **register.action.ts:33–36**, **change-password.action.ts:47–50** y **login.action.ts:28–33** devuelven un mensaje genérico sin registrar la causa.

En **app/** hay not-found.tsx pero no error.tsx ni global-error.tsx, de modo que un fallo de renderizado usa la página por defecto de Next.js.

Impacto

No se aprecia fuga de información técnica, pero los fallos de base de datos o de servicios externos pasan sin diagnóstico y la experiencia ante un error inesperado no está controlada. Es una carencia de observabilidad más que de confidencialidad.

Confidencialidad	Integridad	Disponibilidad
Bajo	**Bajo**	**Bajo**

Probabilidad

Media (3/5): los errores inesperados ocurrirán en producción.

Riesgo

Riesgo = 3 × 2 = 6 (Bajo).

INFORMATIVO 1–2	▶ BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Centralizar el tratamiento de errores:

- Crear app/error.tsx y app/global-error.tsx con mensajes sin detalles internos y un identificador de incidente.
- Registrar siempre la causa en servidor mediante una función común de log, sin devolverla al cliente.

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Media	Medio plazo (1–3 meses)	Bajo (1 día)	Desarrollo backend

Criterio: Mejora seguridad y trazabilidad a la vez.

H10 · Endurecimiento del pipeline de CI y de las dependencias

Severidad	BAJO	**Riesgo (P × I)**	**2 × 3 = 6** de 25
Probabilidad	2 · Baja	**Impacto**	3 · Moderado
Categoría OWASP	A06 · Componentes vulnerables · A08 · Integridad	**CWE**	CWE-1357 · CWE-829
Componente	GitHub Actions y package.json	**Prioridad**	Media · Medio plazo

Descripción

El pipeline ejecuta lint, pruebas y build, pero no incluye ningún control de seguridad. La auditoría de dependencias realizada durante esta revisión sobre package-lock.json detecta vulnerabilidades conocidas, la mayoría en herramientas de desarrollo.

No hay un proceso automático que avise de ellas ni que detecte secretos, por lo que el estado actual depende de revisiones manuales.

Evidencia

.github/workflows/ci.yml:1–20: no declara permissions, fija las acciones por etiqueta (checkout@v4, setup-node@v4) y no ejecuta npm audit, análisis estático ni escaneo de secretos. No existe configuración de Dependabot y los tests e2e con axe no se ejecutan en CI.

npm audit (9 de octubre de 2026) informa de 18 vulnerabilidades: 2 críticas (vitest/tinypool, herramientas de prueba), 14 altas y 2 moderadas. Las altas proceden sobre todo de shadcn (**package.json:38**, un CLI situado en dependencies), del CLI de Prisma, de lint-staged y de eslint-config-next. En ejecución, next 15.5.27 arrastra un postcss anidado vulnerable (moderada).

Impacto

La mayoría de los paquetes afectados no se ejecutan en producción, por lo que no hay una vía de explotación directa. Sí aumentan la superficie de la cadena de suministro y no hay forma de enterarse de una vulnerabilidad nueva en un componente de ejecución como next o next-auth, que sigue en versión beta (5.0.0-beta.32).

Confidencialidad	Integridad	Disponibilidad
Medio	Alto	Bajo

Probabilidad

Baja (2/5) como vía directa de ataque en el contexto actual; creciente con la exposición pública.

Riesgo

Riesgo = 2 × 3 = 6 (Bajo). Se recomienda revisar la severidad si alguna dependencia de ejecución queda afectada.

INFORMATIVO 1–2	▶ BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Añadir controles automáticos y limpiar el árbol de dependencias:

- Ejecutar npm audit --omit=dev en CI con fallo ante severidad alta, y activar Dependabot para npm y Actions.
- Mover shadcn a devDependencies, actualizar vitest, lint-staged y eslint-config-next y planificar la subida de next.
- Añadir gitleaks y CodeQL (o Semgrep), declarar permissions: contents: read y fijar las acciones por hash.
- Incorporar los tests e2e con axe al pipeline y proteger la rama principal.

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Media	Medio plazo (1–3 meses)	Medio (2–3 días)	DevOps / Desarrollo

Criterio: Previene regresiones y avisa de vulnerabilidades nuevas.

H11 · Ausencia de auditoría de eventos de seguridad

Severidad	BAJO	**Riesgo (P × I)**	**2 × 2 = 4** de 25
Probabilidad	2 · Baja	**Impacto**	2 · Bajo
Categoría OWASP	A09 · Fallos de registro y monitorización	**CWE**	CWE-778
Componente	Registro y auditoría	**Prioridad**	Media · Medio plazo

Descripción

La aplicación no registra de forma estructurada los eventos relevantes: inicios de sesión correctos y fallidos, cambios de contraseña o de rol, accesos denegados o borrados de contenido. Sin ellos no se puede detectar un ataque en curso ni reconstruir un incidente.

Evidencia

No existe un módulo de logging: el único registro es el console.error de **upload-photo.action.ts:47**. Ni login.action.ts ni login.service.ts dejan rastro de intentos fallidos o correctos. La especificación excluye la «auditoría de accesos» del alcance actual (**SPEC-010:397**).

Impacto

No añade una vía de ataque, pero anula la detección y la respuesta: los incidentes pasan desapercibidos y el análisis posterior es incompleto.

Confidencialidad	Integridad	Disponibilidad
Bajo	Medio	Bajo

Probabilidad

Baja (2/5) como causa directa; el efecto aparece cuando ocurre cualquier otro incidente.

Riesgo

Riesgo = 2 × 2 = 4 (Bajo).

INFORMATIVO 1–2	▶ BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Definir un catálogo mínimo de eventos y un formato estable:

- Registrar en JSON: marca de tiempo, usuario, IP, acción y resultado, sin contraseñas ni tokens, empezando por login, cambios de contraseña, rol y borrados.
- Enviar los registros a un destino con retención definida y configurar alertas básicas (picos de fallos de login).

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Media	Medio plazo (1–3 meses)	Medio (2 días)	Desarrollo backend

Criterio: Habilita detección y respuesta ante incidentes.

H12 · Observaciones generales

Severidad	INFORMATIVO	**Riesgo (P × I)**	**1 × 1 = 1** de 25
Probabilidad	1 · Muy baja	**Impacto**	1 · Mínimo
Categoría OWASP	Transversal	**CWE**	No aplica
Componente	Proyecto en su conjunto	**Prioridad**	Baja · Largo plazo

Descripción

Observaciones que no son vulnerabilidades pero conviene tener presentes al evolucionar hacia producción.

Evidencia

Surgen del alcance de la revisión y del estado del repositorio:

1. Revisión estática (white-box): sin pruebas dinámicas de penetración sobre un entorno desplegado.
2. No hay SECURITY.md ni modelo de amenazas, y las pruebas existentes no incluyen casos negativos de seguridad.
3. next-auth está en versión beta (5.0.0-beta.32). La aplicación ya incluye páginas legales y de privacidad; falta contrastar con el RGPD los tratamientos que implican a Cloudinary y a OpenStreetMap.

Impacto

Sin impacto directo: son oportunidades de madurez.

Probabilidad

No aplica.

Riesgo

Riesgo = 1 × 1 = 1 (Informativo).

▶ INFORMATIVO 1–2	BAJO 3–7	MEDIO 8–14	ALTO 15–19	CRÍTICO 20–25

Recomendación

Planificar las siguientes acciones de madurez:

- Prueba de penetración dinámica sobre preproducción y repetición de esta auditoría tras la remediación.
- Modelo de amenazas básico (STRIDE), SECURITY.md y revisión del RGPD.
- Seguir la versión estable de Auth.js v5 y planificar la migración.

Prioridad

Nivel	Plazo	Esfuerzo	Responsable
Baja	Largo plazo (3–12 meses)	Variable	Responsable del proyecto

Criterio: Aumenta la confianza global; sin urgencia técnica.

10. Conclusiones ejecutivas

10.1 Valoración global

Accesibilidad 360 presenta un nivel de seguridad **aceptable** para el alcance del Trabajo Fin de Máster. En el análisis no se ha encontrado ninguna vulnerabilidad que permita comprometer la confidencialidad, la integridad o la disponibilidad del sistema dentro del alcance evaluado. Los doce hallazgos son, en esencia, medidas de endurecimiento que cualquier aplicación web debería incorporar antes de abrirse a usuarios reales.

Es importante interpretar correctamente esa valoración. «Aceptable» no significa «terminado»: significa que el diseño es correcto y que las carencias detectadas son corregibles con un esfuerzo reducido. La distancia entre el estado actual y un estado apto para producción se mide en días de trabajo, no en meses ni en rediseños.

> **Veredicto**
> **Accesibilidad 360 puede considerarse segura para el alcance del TFM.** Para un despliegue público con usuarios reales se recomienda cerrar previamente H1–H4 y planificar una prueba de penetración dinámica.

10.2 Fortalezas del proyecto

- **Validación en servidor.** La aplicación no delega la seguridad en el cliente, lo que neutraliza buena parte de las técnicas de manipulación habituales.
- **Autorización centralizada.** Concentrar las decisiones de acceso en un punto reduce el riesgo de rutas olvidadas y facilita su revisión y prueba. Es la razón principal de que A01 quede en riesgo Bajo.
- **Separación de responsabilidades.** La arquitectura en capas permite corregir cada hallazgo de forma local, sin efectos en cascada.
- **Defensa en profundidad parcial.** El layout protegido y la comprobación de sesión de las 13 Server Actions que la requieren compensan que el middleware aún no proteja rutas; falta añadir esa tercera barrera.
- **Ausencia de superficies de alto riesgo.** La única petición saliente va a una URL fija de Nominatim y los archivos subidos se reenvían a Cloudinary sin interpretarse en servidor.

10.3 Debilidades y patrones comunes

Los doce hallazgos no son independientes: se agrupan en tres patrones que explican el estado de seguridad del proyecto y orientan la remediación.

Patrón	Hallazgos	Qué indica
Autenticación y sesión sin endurecer	H1, H2, H5, H6, H7	Los flujos de cuenta funcionan, pero no se han protegido frente a abuso automatizado. Es el dominio donde se concentra el riesgo.
Configuración por defecto	H3, H4, H9	La plataforma funciona con valores por defecto permisivos: sin cabeceras, con un middleware que deja pasar todo y con errores sin normalizar.
Falta de detección y automatización	H8, H10, H11, H12	No existen controles automáticos que detecten un problema ni registros que permitan investigarlo. Es una carencia de proceso más que de código.

10.4 Riesgo residual tras la remediación

Si se aplican las recomendaciones, la exposición esperada pasa de tres categorías OWASP en nivel Medio a ninguna. La tabla compara el riesgo actual con el esperado.

Escenario de amenaza	Hoy	Tras remediar	Medidas que lo reducen
Fuerza bruta y relleno de credenciales	Medio	Bajo	H1, H6, H11
Secuestro o reutilización de sesión	Medio	Bajo	H2, H3, H4

Escenario de amenaza	Hoy	Tras remediar	Medidas que lo reducen
Acceso no autenticado por ruta sin protección	Medio	Bajo	H4, pruebas de acceso
Explotación de XSS o clickjacking	Medio	Bajo	H3, H7
Compromiso por dependencia o pipeline	Bajo	Info	H10
Incidente no detectado	Bajo	Info	H11, H9

10.5 Implicaciones para la decisión

Para el TFM. El trabajo demuestra un diseño de seguridad correcto y una evaluación crítica de sus propias carencias. Las cuatro medidas de severidad Media pueden presentarse como mejoras ya planificadas, con esfuerzo acotado y criterios de cierre medibles.

Para un despliegue real. Antes de publicar la aplicación deben cerrarse H1–H4, activar el registro de eventos básico (H11) y completar la recuperación de contraseña (H5). Además, una aplicación de accesibilidad puede tratar datos que revelan necesidades de sus usuarios, por lo que conviene contrastar el tratamiento con el RGPD.

Para la evolución futura. La mayor ganancia a largo plazo no está en corregir hallazgos concretos sino en institucionalizar los controles: pipeline con verificaciones automáticas, pruebas de seguridad en cada cambio y revisiones periódicas.

10.6 Limitaciones de la conclusión

Esta valoración se apoya en un análisis estático. Los resultados deben confirmarse con pruebas dinámicas en un entorno desplegado, que podrían revelar problemas de configuración de infraestructura no visibles en el código. Por ello se plantea como acción informativa (H12) y se propone repetir la auditoría tras la remediación.

11. Recomendaciones priorizadas

Las acciones se ordenan por plazo y por riesgo reducido por unidad de esfuerzo. Los plazos se cuentan desde la aprobación de este informe.

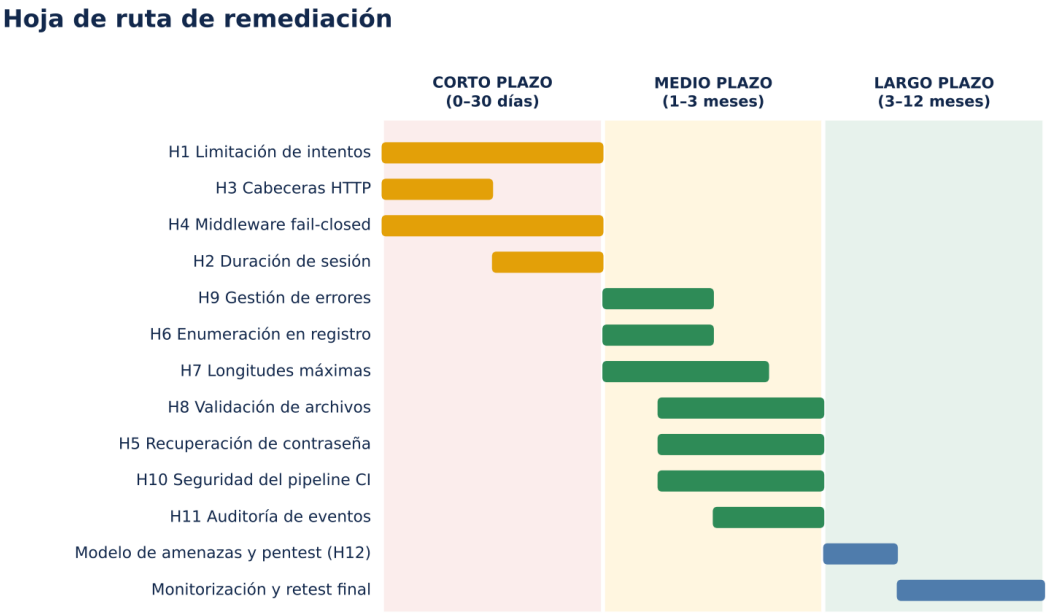


Figura 7. Hoja de ruta de remediación por plazos (corto, medio y largo).

11.1 Corto plazo (0–30 días)

Objetivo: eliminar todos los hallazgos de severidad Media y situar la aplicación en condiciones de salir a producción.

ID	Acción	Esfuerzo	Responsable	Criterio de aceptación
H1	Limitar intentos de autenticación por IP y por cuenta con espera progresiva	0,5–1 d	Backend	Ráfaga de 50 intentos devuelve 429
H3	Añadir cabeceras de seguridad (CSP en modo informe, HSTS, nosniff, frame-ancestors)	0,5–1 d	Backend / DevOps	Cabeceras presentes en todas las rutas; prueba en CI
H4	Implementar authorized() con denegación por defecto y pruebas de acceso	1 d	Backend	Ruta privada sin sesión redirige a /login
H2	Fijar maxAge de 8 h, refrescar el rol y añadir tokenVersion	1 d	Backend	Token caduca y cambios de rol surten efecto

11.2 Medio plazo (1–3 meses)

Objetivo: completar el endurecimiento y dotar al proyecto de capacidad de detección.

ID	Acción	Esfuerzo	Responsable	Criterio de aceptación
H5	Implementar recuperación de contraseña con token de un solo uso y caducidad	2–3 d	Full-stack	Flujo completo con respuesta uniforme
H6	Unificar respuestas de registro e igualar tiempos	0,5 d	Backend	Misma respuesta con correo existente o nuevo
H7	Acotar longitudes máximas en todos los campos y limitar tamaño del cuerpo	1 d	Backend	Entradas sobredimensionadas rechazadas
H8	Validar archivos por firma, renombrar, almacenar fuera de la raíz pública	2 d	Backend	Archivo con extensión falsa es rechazado

ID	Acción	Esfuerzo	Responsable	Criterio de aceptación
H9	Manejador global de errores con formato único e identificador de correlación	1 d	Backend	Sin trazas internas en respuestas
H10	Análisis de dependencias, secretos y SAST en CI; permisos mínimos	2–3 d	DevOps	Pipeline falla ante vulnerabilidad alta
H11	Registro estructurado de eventos de seguridad y alertas básicas	2 d	Backend	Eventos de login y accesos denegados trazables

11.3 Largo plazo (3–12 meses)

Objetivo: consolidar un programa de seguridad sostenible y verificar de forma independiente la remediación.

ID	Acción	Esfuerzo	Responsable	Criterio de aceptación
H12	Prueba de penetración dinámica sobre preproducción	3–5 d	Auditor externo	Informe sin hallazgos Altos o Críticos
H12	Modelo de amenazas (STRIDE) y archivo SECURITY.md	1–2 d	Responsable	Documento versionado en el repositorio
H12	Revisión de cumplimiento RGPD y evaluación de impacto si procede	Variable	Responsable / DPO	Registro de tratamientos documentado
—	Monitorización continua y gestión de vulnerabilidades en dependencias	Continuo	DevOps	Alertas revisadas con periodicidad definida
—	Reauditoría completa tras la remediación	2–3 d	Auditor	Todos los hallazgos cerrados o aceptados

11.4 Indicadores de éxito

Indicador	Hoy	Objetivo a 90 días
Hallazgos de severidad Media o superior abiertos	4	0
Categorías OWASP con exposición Media	3	0
Cabeceras de seguridad obligatorias presentes	0 de 6	6 de 6
Eventos de seguridad registrados de forma estructurada	No	Sí, con alertas
Controles de seguridad automáticos en CI	0	3 (dependencias, secretos, SAST)

Anexo A. Glosario

Término	Definición
Rate limiting	Limitación del número de peticiones que un origen puede realizar en una ventana de tiempo.
CSP	Content Security Policy: cabecera que restringe los orígenes desde los que se pueden cargar scripts y otros recursos.
HSTS	HTTP Strict Transport Security: obliga al navegador a usar siempre HTTPS con el sitio.
SAST / DAST	Análisis estático / dinámico de seguridad de aplicaciones.
CWE	Common Weakness Enumeration: catálogo público de tipos de debilidad de software.

Anexo B. Lista de verificación para la reauditoría

Lista para confirmar el cierre de cada hallazgo tras la remediación. Marcar cuando la prueba haya superado el criterio indicado.

	ID	Comprobación	Resultado esperado
☐	**H1**	Ráfaga de 50 logins fallidos	Respuesta 429 y espera
☐	**H2**	Vigencia del token y cambio de rol	maxAge ≤ 8 h y rol refrescado
☐	**H3**	Seis cabeceras en todas las rutas	Presentes y sin errores de CSP
☐	**H4**	Ruta privada sin sesión	Redirige a /login o devuelve 401
☐	**H5**	Flujo de recuperación completo	Token de un solo uso y caducidad
☐	**H6**	Registro con correo existente y nuevo	Misma respuesta y tiempo similar
☐	**H7**	Cadena de 1 MB en campos de texto	Rechazo con error 4xx
☐	**H8**	Archivo con extensión falsa	Rechazado por firma
☐	**H9**	Provocar errores 4xx y 5xx	Formato único sin trazas
☐	**H10**	Pipeline con tres controles de seguridad	Falla ante vulnerabilidad alta
☐	**H11**	Login fallido en el registro de auditoría	Evento con usuario, IP y resultado

Anexo C. Control del documento

Datos de identificación del informe, historial de versiones y condiciones de uso. Debe conservarse junto con el documento en cualquier distribución.

Identificación

Título	Informe de auditoría de seguridad · Accesibilidad 360
Cliente / destinatario	Equipo del proyecto Accesibilidad 360 y tribunal del TFM Desarrollo con IA (Big School)
Auditor	Jorge Serna

Tipo de revisión	Revisión estática (white-box) de código, configuración, arquitectura, documentación y pruebas
Repositorio	sernalovic/accesibilidad-360 · commit 5722e3e · Next.js 15, Auth.js v5, Prisma, Cloudinary
Referencias	OWASP Top 10 (2021) · OWASP ASVS · CWE · escala propia de riesgo P × I (apartado 4)
Clasificación	Confidencial

Historial de versiones

Versión	Fecha	Autor	Descripción
1.0	Septiembre 2026	Jorge Serna	Informe inicial de resultados (resumen de hallazgos y recomendaciones).
2.0	9 octubre 2026	Jorge Serna	Informe ampliado: arquitectura, matriz de riesgos, ficha detallada por hallazgo, gráficos, conclusiones ejecutivas y hoja de ruta priorizada.

Aviso y limitaciones

Este informe refleja el estado del proyecto en el momento de la revisión y se basa exclusivamente en un análisis estático. No sustituye una prueba de penetración dinámica ni garantiza la ausencia de otras vulnerabilidades. Las valoraciones de severidad, probabilidad e impacto siguen la escala definida en el apartado 4 y pueden reevaluarse si cambian el contexto de despliegue, los datos tratados o la exposición de la aplicación.